

Reviewer Pack — Non-Abelian Fourier Coefficient Spread Inversely Proportiona...

Entry 2bf1ffaff7e5 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-12 19:42:26 UTC

Non-Abelian Fourier Coefficient Spread Inversely Proportional to Resolution Proof Length for Tseitin Formulas

Entry ID: 2bf1ffaff7e5 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-12 19:42:26 UTC

1. Conjecture

Field A (mathematical branch): Non-Abelian Harmonic Analysis **Field B** (complexity object): Resolution Proof Length for Tseitin Formulas

Statement:

For a Tseitin formula Φ over a connected planar graph G , the spread of non-abelian Fourier coefficients ($\max_k |F(k)| - \min_k |F(k)|$) of its truth table is inversely proportional to the minimal resolution proof length of Φ , with proportionality constant $\Theta(\sqrt{n})$.

Rationale (proposer's reasoning):

Non-abelian Fourier analysis captures symmetries in boolean functions' truth tables, while Tseitin formulas' resolution complexity is tied to graph structure. The conjecture links algebraic structure (non-abelian Fourier coefficients) to combinatorial proof complexity, potentially revealing hidden invariants in planar satisfiability.

Taxonomy category: AVG_TO_WORST_CASE (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 5563d5602cb2dd2a

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	UNCERTAIN	SAFE
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from itertools import permutations

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_planar_graph(n):
        # Simple planar graph generation (not exhaustive)
        if n == 3:
            return [(0, 1), (1, 2), (2, 0)]
        elif n == 4:
            return [(0, 1), (1, 2), (2, 3), (3, 0), (0, 2)]
        else:
            raise ValueError("Unsupported graph size for this simple generator")

    def tseitin_formula(graph):
        # Construct Tseitin formula from a planar graph
        n = len(graph)
        literals = [f"x{i}" for i in range(n)]
        clauses = []
        for u, v in graph:
            clauses.append([literals[u], f"~{literals[v]}"])
            clauses.append([f"~{literals[u]}", literals[v]])
        return clauses

    def non_abelian_fourier_coefficients(clauses):
        n = len(clauses)
        F = [0] * (2 ** n)
        for perm in permutations(range(n)):
            sign = 1
            for i, clause in enumerate(clauses):
                if all(perm[j] == int(lit[1:]) - 1 for lit in clause if lit[0] != '~'):
                    continue
                elif any(perm[j] == int(lit[1:]) - 1 for lit in clause if lit[0] == '~'):
                    sign *= -1
            F[sum(1 if i == j else 0 for i, j in enumerate(perm))] += sign
        return F
```

```

def resolution_length(clauses):
    # Simple DPLL with clause learning (not exhaustive)
    stack = []
    learned_clauses = set()
    while clauses:
        literal = random.choice([c[0] for c in clauses if c[0][0] != '~'] + [f"~{c[0]}" for c in clauses if c[0][0] == '~'])
        if literal.startswith('~'):
            literal = literal[1:]
            polarity = False
        else:
            polarity = True
        stack.append((literal, polarity))
        while stack:
            lit, pol = stack.pop()
            if lit in learned_clauses:
                continue
            found = False
            for i, clause in enumerate(clauses):
                if literal in clause:
                    clauses[i].remove(literal)
                    if not clauses[i]:
                        return 0
                    if polarity != (lit[0] == '~'):
                        stack.extend([(c, True) for c in clauses[i]])
                    else:
                        stack.extend([(c, False) for c in clauses[i]])
                    found = True
                    break
            if not found:
                learned_clauses.add(lit)
    return len(learned_clauses)

n = random.choice([5, 10, 15, 20, 30, 40])
graph = generate_planar_graph(n)
clauses = tseitin_formula(graph)
F = non_abelian_fourier_coefficients(clauses)
coefficient_spread = max(abs(x) for x in F) - min(abs(x) for x in F)
proof_length = resolution_length(clauses)

if coefficient_spread * proof_length == 0:
    return {
        "metric_name": "spread_times_proof_length",
        "metric_value": None,
        "instances_tested": 1,
        "conjecture_holds": False,
        "counterexample": "division_by_zero"
    }

expected = math.sqrt(n)
ratio = coefficient_spread * proof_length / expected

return {
    "metric_name": "spread_times_proof_length",
    "metric_value": ratio,
    "instances_tested": 1,
    "conjecture_holds": abs(ratio - 1) < 0.1, # Allow some tolerance

```

```

        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2*i + 3 for i in range(5, 8)] # Default to first

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\"}, \"metric_value\": {result['metric_value']}")
        results.append(result)

    if all(r["conjecture_holds"] for r in results):
        support_fraction = 1.0
    else:
        support_fraction = sum(r["conjecture_holds"] for r in results) / len(results)

    mean_value = sum(r["metric_value"] for r in results if r["metric_value"] is not None) / sum(1 for r in results if r["metric_value"] is not None)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results if r["metric_value"] is not None) / len(results))

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{result['counterexample']}\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE insufficient_data")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0e8e9054.py", line 117, in <module>
    result = run_trial(seed)
              ~~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0e8e9054.py", line 85, in run_trial
    graph = generate_planar_graph(n)
             ~~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0e8e9054.py", line 27, in generate_planar_graph
    raise ValueError("Unsupported graph size for this simple generator")
ValueError: Unsupported graph size for this simple generator

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed due to unsupported graph size, preventing data collection to verify proportionality claim | next: Implement a more robust planar graph generator supporting larger n values

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	108240
2	propose	ollama_remote	qwen3:8b	0	0	112104
3	preregistration	ollama_remote	qwen3:8b	0	0	24153
4	novelty	ollama_remote	qwen3:8b	0	0	20910
5	novelty	ollama_remote	qwen3:8b	0	0	14627
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19981
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15033
8	judge	ollama_remote	qwen3:8b	0	0	12786

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 327834 ms total latency. Provider mix: {'ollama_remote': 8}

(full prompt+response transcripts available in `research/audit/2bf1ffaff7e5.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/2bf1ffaff7e5.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/2bf1ffaff7e5.tar.gz` (if generated)